Flyweight Pattern

1. Problem Statement: Millions of Similar Game Objects

Modern games frequently manage *huge numbers* of similar objects:

- A bullet-hell shooter with **200,000 bullets** on screen over a few seconds.
- A tiled world with **millions of tiles** across chunks.
- An open-world forest with **hundreds of thousands of trees** sharing the same meshes/-textures.
- Particle systems with **hundreds of thousands of particles**.

Naïvely, every object holds a texture/mesh reference, shader parameters, stats (damage, speed), animation data, etc. Even if much of that data is *identical* across objects of the same kind, we end up **duplicating intrinsic state** in memory. Consequences:

- High memory footprint ⇒ paging, poor cache locality, GC pressure (on managed runtimes).
- CPU gets busy moving/collecting memory rather than simulating/rendering.
- Data bloat reduces bandwidth for streaming and save/load.

We need a way to *share* the heavy, identical state across many objects, while keeping only the varying bits (position, rotation, velocity, health) per instance.

2. Forces, Constraints, and Goals

- **Large-N** : many game entities of few *types* (e.g., 12 bullet types, 6 tree types).
- **Intrinsic vs. extrinsic** : immutable shared state (textures, meshes, base parameters) vs. per-instance state (transform, current HP, lifetime).
- **Memory & locality** : put large immutable data into shared objects so per-instance arrays are compact and cache-friendly.
- **Thread-safety** : shared state should be immutable so many threads can read it safely.
- **Determinism** : separating shared/instance data makes simulation more predictable and testable.

3. Naïve Alternatives and Why They Fail

3.1 Copy everything per instance

Easiest to implement: each bullet/tree/tile stores sprite/mesh/stats. This works at small scale, but:

- Memory explodes linearly with instance count.
- GC or allocator overhead spikes; poor CPU cache utilization.

3.2 Global singletons of textures/meshes, but duplicate params

Better, but you still duplicate "semi-constant" parameters (base damage, base speed, collision mask). You also end up sprinkling global lookups everywhere and mixing responsibilities.

3.3 Object Pooling alone

Pooling reduces *churn* but not *size*. If pooled objects are still large because they carry intrinsic data, you save allocation time but keep the memory bloat.

3.4 ECS only, without sharing heavy state

Entity-Component-System helps layout, but if each entity's components duplicate identical big blobs (e.g., per-entity copy of a 200KB mesh/material), the memory problem remains.

4. The Flyweight Pattern: Core Idea

Flyweight separates an object into:

- **Intrinsic state** (shared): immutable, identical across many instances (e.g., *BulletType* sprite, *TileType* texture & collision rules, *TreeType* mesh & material).
- **Extrinsic state** (per-instance): unique, stored *outside* the flyweight (position, rotation, velocity, lifetime, owner, current animation frame).

A **Flyweight Factory** hands out shared flyweights keyed by type parameters (e.g., "GREEN_BULLET, speed=600, dmg=10"). The game loop passes extrinsic state to the flyweight's operations (e.g., `render(extrinsic)`), or each instance holds a *lightweight reference* to its flyweight.

Why it works in games Most entities are drawn from a *small catalog of types*. Sharing immutable assets and base parameters yields massive memory wins and better cache behavior; per-instance arrays (SoA) remain tiny and hot.

5. Design Heuristics (Game Dev)

- Keep flyweights **immutable**. They can be shared across threads (render/sim).
- **Key** flyweights by the *minimal* set of attributes that define a type (texture set, material, base stats). Avoid over-keying (causes flyweight explosion).
- Move per-instance state into **contiguous arrays** (ECS-friendly; SoA layout: positions[], rotations[], velocities[]).
- When rendering, pass extrinsic state to the flyweight (`draw(type, pos, rot)`), or store a *handle* (index or ref) to the type.
- Combine with **Object Pool** for short-lived instances (bullets/particles) to reduce allocation churn.
- Use **factories** to manage caches (weak refs or explicit eviction if catalog is large or streaming).

6. Concrete Java Example (bullet types, trees, tiles)

Below is an intentionally compact Java sketch (engines will differ, but the structure generalizes to C#/C++/ECS).

6.1 Intrinsic Types (flyweights)

Listing 1: Flyweights: immutable type catalogs

```java
/* TextureId/MaterialId would be engine-specific handles */
public record TextureId(String name) {}
public record MeshId(String name) {}

public final class BulletType {
  public final TextureId texture;
  public final double baseDamage;
  public final double baseSpeed;
  public final int spriteW, spriteH;

  public BulletType(TextureId texture, double baseDamage, double baseSpeed,
                    int spriteW, int spriteH) {
    this.texture = texture;
    this.baseDamage = baseDamage;
    this.baseSpeed = baseSpeed;
    this.spriteW = spriteW; this.spriteH = spriteH;
  }
}

public final class TreeType {
  public final MeshId mesh;
  public final TextureId bark, leaves;
  public final double trunkRadius;
  public final double baseWindResponse; // shader param

  public TreeType(MeshId mesh, TextureId bark, TextureId leaves,
                  double trunkRadius, double baseWindResponse) {
    this.mesh = mesh; this.bark = bark; this.leaves = leaves;
    this.trunkRadius = trunkRadius; this.baseWindResponse =
        baseWindResponse;
  }
}

public final class TileType {
  public final TextureId atlas;
  public final int atlasX, atlasY; // UV tile coords
  public final boolean blocksMovement;
  public final boolean blocksVision;

  public TileType(TextureId atlas, int atlasX, int atlasY,
                  boolean blocksMovement, boolean blocksVision) {
    this.atlas = atlas; this.atlasX = atlasX; this.atlasY = atlasY;
    this.blocksMovement = blocksMovement; this.blocksVision = blocksVision;
  }
}
```

6.2 Flyweight Factories (caching by key)

Listing 2: Factories: ensure sharing, not duplication

```
import java.util.*;

public final class BulletTypeFactory {
  private final Map<String,BulletType> cache = new HashMap<>();

  public BulletType get(TextureId tex, double dmg, double speed, int w, int
      h) {
    String key = tex.name()+"|"+dmg+"|"+speed+"|"+w+"|"+h;
    return cache.computeIfAbsent(key, k -> new BulletType(tex, dmg, speed,
        w, h));
  }

  public int size() { return cache.size(); }
}

public final class TreeTypeFactory {
  private final Map<String,TreeType> cache = new HashMap<>();
  public TreeType get(MeshId mesh, TextureId bark, TextureId leaves,
                      double radius, double wind) {
    String key = mesh.name()+"|"+bark.name()+"|"+leaves.name()+"|"+radius+"
        |"+wind;
    return cache.computeIfAbsent(key, k -> new TreeType(mesh, bark, leaves,
         radius, wind));
  }
}

public final class TileTypeFactory {
  private final Map<String,TileType> cache = new HashMap<>();
  public TileType get(TextureId atlas, int x, int y, boolean blockM,
     boolean blockV) {
    String key = atlas.name()+"|"+x+"|"+y+"|"+(blockM?1:0)+"|"+(blockV?1:0)
        ;
    return cache.computeIfAbsent(key, k -> new TileType(atlas, x, y, blockM
        , blockV));
  }
}
```

6.3 Extrinsic State (lightweight instances)

Listing 3: Per-instance data holds only what varies

```
/* A bullet instance points to a shared BulletType; all other fields are
   extrinsic */
public final class Bullet {
  public BulletType type;   // shared flyweight (intrinsic)
  public float x, y;        // extrinsic
  public float vx, vy;      // extrinsic
  public float lifetime;    // extrinsic
  public int ownerId;       // extrinsic (who fired)

  public void update(float dt) {
    x += vx * dt; y += vy * dt; lifetime -= dt;
  }
}
```

```java
/* A tree instance: transform + wind phase; shares TreeType mesh/materials
   */
public final class Tree {
  public TreeType type;
  public float x, y, z;
  public float scale;
  public float windPhase;
}

/* A tile instance: just a reference to TileType plus coordinates */
public final class Tile {
  public TileType type;
  public int gx, gy; // grid coords
}
```

6.4 Rendering with extrinsic state

Listing 4: Renderer consumes extrinsic + flyweight

```java
/* Pseudo rendering interfaces */
interface Graphics {
  void drawSprite(TextureId tex, int spriteW, int spriteH, float x, float y
     , float rotation);
  void drawMesh(MeshId mesh, TextureId bark, TextureId leaves, float x,
     float y, float z, float scale, float wind);
  void drawTile(TextureId atlas, int atlasX, int atlasY, int gx, int gy);
}

public final class RenderSystem {
  public void drawBullet(Graphics g, Bullet b, float rotation) {
    BulletType t = b.type; // shared
    g.drawSprite(t.texture, t.spriteW, t.spriteH, b.x, b.y, rotation);
  }
  public void drawTree(Graphics g, Tree t) {
    g.drawMesh(t.type.mesh, t.type.bark, t.type.leaves,
               t.x, t.y, t.z, t.scale, t.windPhase * t.type.
                  baseWindResponse);
  }
  public void drawTile(Graphics g, Tile tile) {
    TileType t = tile.type;
    g.drawTile(t.atlas, t.atlasX, t.atlasY, tile.gx, tile.gy);
  }
}
```

6.5 Memory impact: a quick, concrete comparison

Assume:

- Each `BulletType` (intrinsic) $\approx$ 96 bytes (handles, base params).
- Each `Bullet` (extrinsic) $\approx$ 32 bytes (pos/vel/lifetime/owner + ref).

Without flyweight: duplicate 96B for every bullet. For 200,000 bullets $\Rightarrow$ $\sim$19.1 MB intrinsic duplication *alone* (200k $\times$ 96B) + extrinsic $\sim$6.4 MB $\Rightarrow$ $\sim$25.5 MB.
With flyweight: share, e.g., 12 bullet types $\Rightarrow$ 12 $\times$ 96B = 1,152B ($\sim$1.1KB) + extrinsic 6.4 MB $\Rightarrow$ $\sim$6.4 MB total.

You save ≈**19 MB** and, more importantly, the hot per-instance arrays (positions/velocities) are compact and cache-friendly.

7. Valid Doubts (and Answers)

Why not modify legacy assets to carry less data per instance?
Because assets (meshes, textures, base parameters) are inherently shared across many instances; baking them into instances duplicates data and couples authoring pipelines to runtime layout. Flyweight cleanly separates authoring-time shared data from runtime instance data.

Is flyweight the same as object pooling?
No. Pooling reduces allocation *churn*; flyweight reduces *per-instance size* by sharing immutable data. They complement each other.

.

8. Pitfalls and How to Avoid Them

- **Mutable flyweights**: if you mutate shared state at runtime, all instances change (often unintentionally). Make flyweights immutable; create a new type instead.
- **Key explosion**: if you include animation frame or random seed in the key, you'll create tons of nearly-identical flyweights. Key only on truly shared attributes.
- **Hidden duplication**: ensure your factory is actually reused. Don't create separate factories per scene unless they intentionally share caches.
- **Leaking extrinsic into flyweights**: do not stash instance-specific values (position, owner) inside the flyweight.
- **Over-abstracting**: sometimes a simple shared material/mesh table is enough; don't force pattern ceremony if the engine already does it.

9. Testing Strategy (Game-Focused)

- **Factory tests**: same key ⇒ same instance (reference equality); different key ⇒ distinct instance.
- **Immutability**: verify flyweights can't be modified after creation.
- **Memory profile**: before/after memory snapshots (or heap histograms) with large-N entity counts.
- **Render determinism**: rendering the same list of (type, transform) pairs yields identical draw calls.
- **Concurrency**: multiple threads can read flyweights safely (no data races).

10. Where Flyweight Shows Up in Common Engine Patterns

- **Tile maps**: millions of tiles referencing ∼100 *TileType*s (atlas UV + collision flags).
- **Vegetation/props**: *TreeType*/*RockType* share meshes/materials; instances store transforms.

- **Bullets/projectiles**: *BulletType* with textures/base stats; instances hold position/velocity.
- **Particles**: often SoA arrays with shared *ParticleMaterial*/*EmitterType*.
- **UI glyphs**: font *Glyph* metrics shared; each character instance stores only quad transform.

11. Relationship to Nearby Patterns

- **Object Pool**: orthogonal; pool reduces allocation churn, flyweight reduces per-instance size.
- **Prototype**: cloning for new instances; with flyweight you clone/extract only extrinsic data, not heavy intrinsic parts.
- **Singleton/Service Locator**: sometimes used for the flyweight factory, but prefer explicit factories for testability.

12. When to Use / When Not to Use

Use Flyweight when

- You have **large numbers** of objects from a **small catalog** of types.
- Intrinsic state is big and **immutable**, and per-instance state is small.
- Memory and cache locality matter (they always do in games).

Avoid (or limit) Flyweight when

- Each instance has unique heavy data (e.g., a unique mesh per hero).
- Intrinsic state must mutate per instance (then it's not intrinsic).
- The catalog of types is as large as the number of instances (no sharing benefit).

13. Mini Walkthrough: Applying Flyweight to a Bullet Hell

1. Identify intrinsic fields: sprite/texture, base damage/speed, collision mask.
2. Build a **BulletTypeFactory** keyed by those fields. Make `BulletType` immutable.
3. Store per-bullet arrays for (x, y), (v_x, v_y), lifetime, owner, and a *type handle*.
4. In the update/render loop, use the handle to fetch the shared flyweight and pass the instance's transform/lifetime for behavior and draw.
5. Optional: add an **Object Pool** for bullets to reuse instance slots.

14. Takeaway

Flyweight is a **memory and cache locality** pattern first, an OO pattern second. In games, the payoff is huge: you squeeze millions of entities into memory comfortably and keep hot loops tight by pushing immutable, repeated data into shared flyweights. Pair it with ECS/pooling for best results, and keep flyweights truly immutable and properly keyed.